Entra ID Governance — Access Reviews & Entitlement Management

Implementation Solution Guide

Version 1.0.0 · 2026-07-09

Contents

Executive Summary — Entra ID Governance: Access Reviews & Entitlement Management 5

The problem . . . 5

The solution . . . 5

What's included . . . 5

Business value . . . 5

Effort and boundary . . . 6

1. Solution Overview — Entra ID Governance: Access Reviews & Entitlement Management 6

What this solution is . . . 6

How the technology works . . . 6

The standard deployment model . . . 7

What is in scope . . . 7

What is out of scope . . . 7

Authentication and connections used . . . 8

Limitations (be honest with buyers) . . . 8

2. Requirements Guide — Entra ID Governance: Access Reviews & Entitlement Management 8

2.1 Entra ID Governance / P2 licensing for the users in scope . . . 8

2.2 Identity Governance administrative role . . . 9

2.3 The resources to govern exist (groups, apps, sites) . . . 9

2.4 A pilot group and named requestors / approvers / reviewers . . . 9

2.5 Manager attribute populated (for manager-as-approver / reviewer and mover workflows) . . . 9

2.6 Employee lifecycle attributes for workflows (joiner/mover/leaver) . . . 9

2.7 Guest / external collaboration settings decided . . . 10

2.8 Microsoft Graph PowerShell toolchain . . . 10

Checklist . . . 10

Global Requirements — Entra ID Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 11

G2. Entra ID licensing (P1, P2 where noted) . . . 11

G3. Administrative roles . . . 11

G4. Break-glass (emergency access) accounts . . . 11

G5. Healthy on-premises Active Directory . . . 11

G6. An Entra Connect host server . . . 11

G7. UPN / domain hygiene . . . 12

G8. Network access to Microsoft cloud endpoints . . . 12

3. Implementation Guide — Entra ID Governance: Access Reviews & Entitlement Management

Step 1 — Assess and baseline the current state . . . 13

Step 2 — Create the governance catalog . . . 13
Step 3 — Create the pilot access package . . . 13
Step 4 — Add the employee assignment policy (request + approval + expiry) . . . 14
Step 5 — Add the guest (time-bound external) assignment policy . . . 14
Step 6 — Create the recurring access reviews . . . 14
Step 7 — Create the lifecycle workflows (joiner / leaver) . . . 15
Step 8 — Validate the pilot (the gates) . . . 16
Step 9 — Expand to production in waves . . . 16
5. Testing & Validation — Entra ID Governance: Access Reviews & Entitlement Management 17
Test matrix . . . 17
Automated checks . . . 18
Evidence to capture (for audit / recertification) . . . 18
Exit criteria (before Expand) . . . 18
6. Troubleshooting — Entra ID Governance: Access Reviews & Entitlement Management 19
The Identity Governance blades show a licensing banner / creation is blocked . . . 19
An access package isn't requestable in My Access . . . 19
Approval never arrives / request sits pending . . . 19
A granted assignment has no expiry (access is permanent) . . . 19
An access review Deny didn't remove access . . . 19
An access review removed access it shouldn't have . . . 20
A guest can't request the guest package . . . 20
A lifecycle workflow didn't fire on schedule . . . 20
A leaver ran but access/sessions remained . . . 20
Editing / rebuilding a package broke existing grants . . . 20
7. Best Practices — Entra ID Governance: Access Reviews & Entitlement Management 21
Entitlement management (catalogs & packages) . . . 21
Access reviews . . . 21
Lifecycle workflows . . . 22
Guest governance . . . 22
Change safety . . . 22
Hygiene . . . 23
Where to go next (adjacent catalog areas) . . . 23
8. Deliverables Summary — Entra ID Governance: Access Reviews & Entitlement Management 23
What you receive . . . 23
Outcomes . . . 24
Assumptions & boundaries . . . 25

Appendix A — PowerShell Scripts 26

Executive Summary — Entra ID Governance: Access Reviews & Entitlement Management

The problem

Access sprawl is the default outcome in any growing tenant. Guests invited for one project stay forever, employees accumulate group and app access as they change roles, and standing membership of privileged and sensitive groups is never re-checked. When an auditor asks "who has access to this and why — and who approved it," the honest answer is usually a spreadsheet assembled by hand the night before. The cost is real: failed access recertifications, over-privileged accounts that widen the breach blast radius, and IT time burned on manual joiner/mover/leaver tasks.

The solution

This solution operationalizes **Microsoft Entra ID Governance** so access is **requested, approved, time-bound, reviewed, and revoked automatically** across three levers:

1. **Entitlement management** — publish **access packages** (bundles of groups, apps, and SharePoint sites) in **catalogs**, governed by **approval and lifecycle policies**, so users request access and it is granted with an expiry instead of assigned by hand and left forever.
1. **Access reviews** — recurring recertification of **group members, application assignments, and guests**, with reviewers who approve or remove access and an auto-apply decision, producing the audit evidence recertification demands.
1. **Lifecycle workflows** — automated **joiner / mover / leaver** tasks (enable account, send welcome, remove group memberships, disable and revoke on exit) triggered by employee lifecycle attributes.

Delivery follows a disciplined **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** model: build and prove one catalog, one access package, one review, and one workflow against a **pilot** group first, confirm explicit pass/fail gates, then expand to the full scope — each step reversible.

What's included

Full documentation (overview, requirements, ordered implementation, testing matrix, troubleshooting, best practices) plus **five PowerShell (Microsoft Graph) tools** that inventory the current governance posture, validate the named objects are in place, export audit/recertification evidence, report on stale guest access, and preview lifecycle-workflow changes. Every tool is **read-only or `-WhatIf` by default**, uses least-privilege Graph scopes, and stores **no credentials**.

Business value

- **Least privilege maintained automatically** — access expires and is re-certified instead of accumulating.
- **Audit-ready** — every grant has a request, an approver, and a review decision on record.

- **Fewer manual tickets** — joiner/mover/leaver tasks and access requests are automated and delegated.
- **Repeatable and reversible** — a pilot-first, gated method with a documented rollback for every change.

Effort and boundary

A competent identity admin can stand up the pilot (one catalog, package, review, and workflow) in a few focused sessions, then expand in waves. This is a **technical implementation** guide: it provides the controls and the automation. **Which** roles, groups, and guests must be reviewed, **how often**, and **who** the approvers and reviewers are remain the customer's governance, security, and compliance decisions.

> ***Licensing note:*** *entitlement management, access reviews, and lifecycle workflows require* ***Microsoft Entra ID Governance*** *(or the features formerly covered by* ***Entra ID P2****). Lifecycle workflows and some entitlement-management capabilities are part of the* ***Entra ID Governance*** *SKU specifically. *Verify current licensing and feature availability in your tenant before you begin.**

1. Solution Overview — Entra ID Governance: Access Reviews & Entitlement Management

What this solution is

This solution deploys the three pillars of **Microsoft Entra ID Governance** that together automate the access lifecycle:

- **Entitlement management** — a self-service, policy-governed way to grant access. You bundle resources (groups, enterprise apps, SharePoint sites) into an **access package**, publish it in a **catalog**, and attach **assignment policies** that control who can request it, who approves, and how long the grant lasts. Users request access from **My Access**; approved grants are **time-bound** and expire automatically.
- **Access reviews** — scheduled recertification. A review targets the members of a group, the users assigned to an application, or the guests in the tenant, and asks named **reviewers** (or the users themselves) to confirm access is still needed. Decisions can **auto-apply** so removals actually happen.
- **Lifecycle workflows** — event-driven automation for the employee lifecycle. A **workflow** runs a set of **tasks** (enable account, add to groups, send a welcome email, request a package, remove memberships, disable account, revoke sessions) triggered by joiner, mover, or leaver conditions based on attributes like `employeeHireDate` and `employeeLeaveDateTime`.

How the technology works

All three features live under **Identity Governance** in the Microsoft Entra admin center (`https://entra.microsoft.com` → **Identity governance**) and are backed by the Microsoft Graph `identityGovernance` API set. The building blocks nest as follows:

- **Catalog** → a container that owns **resources** and **access packages**. Catalogs let you delegate

administration to resource owners without giving them tenant-wide rights.

- **Access package** → the requestable unit. It references **resource roles** (e.g. Member of a group, a specific app role, a SharePoint site role) drawn from its catalog.
- **Assignment policy** → attached to an access package; defines the **requestor scope** (specific users, all members, or specific connected external organizations for guests), the **approval** settings (one or two stages, approvers, justification), and the **lifecycle** (expiry, access review on renewal).
- **Access review definition** → the recurring schedule, scope, reviewers, and decision behavior (auto-apply, what to do if reviewers don't respond).
- **Workflow** → the ordered list of **tasks** plus the **execution conditions** (trigger and scope).

The standard deployment model

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand*** *— inventory current access sprawl and decide the first scope (Assess); build one catalog, one access package with an approval policy, one access review, and one lifecycle workflow (Configure); target them at a small* ***pilot*** *group first (Pilot); prove the pass/fail gates — request→approve→expire, review→auto-apply removal, workflow runs (Validate); then roll the pattern out to the full set of resources, groups, apps, and guests in waves (Expand). Every production change has a rollback.*

What is in scope

- One **governance catalog** with resources added.
- One or more **access packages** with **approval + lifecycle policies** (time-bound, review-on-renewal).
- **Recurring access reviews** for a group, an application's assignments, and **all guests**.
- **Lifecycle workflows** for **joiner** (onboarding) and **leaver** (offboarding), plus the pattern for **mover**.
- **Guest governance** — reviewing external users and enforcing **time-bound external access** via package expiry and guest access reviews.
- Read-only **reporting** for audit and recertification evidence via the included scripts.

What is out of scope

- **Privileged Identity Management (PIM)** for just-in-time **role** activation and role-assignment reviews is a **separate, adjacent solution** (Entra ID Governance — PIM & Access Reviews). This solution governs **resource and app access**, not privileged directory-role activation.
- **Conditional Access** policy design (a separate Zero Trust solution).
- The **decision** of which resources, how often, and who approves/reviews — those are governance, compliance, and business-owner determinations. This guide provides the technical controls.

Authentication and connections used

• **Microsoft Entra admin center** (`https://entra.microsoft.com`) → **Identity governance** for all

configuration.

• **Microsoft Graph** for the automation scripts, via `Connect-MgGraph` with least-privilege scopes

(`EntitlementManagement.Read.All`, `AccessReview.Read.All`, `LifecycleWorkflows.Read.All`, `User.Read.All`). No credentials or secrets are stored in the scripts.

• **My Access** portal (`https://myaccess.microsoft.com`) — where end users request packages and

reviewers act on reviews.

Limitations (be honest with buyers)

• **Licensing gate:** entitlement management, access reviews, and lifecycle workflows require **Entra ID

Governance **/ P2-class licensing. Lifecycle workflows in particular are part of the** Entra ID Governance** add-on. *Verify current SKU coverage and any per-user licensing rules against Microsoft's current documentation and your tenant.*

• **Auto-apply actually removes access.** An access review with auto-apply, or an access package expiry,

will **remove memberships/assignments** — always pilot with report-style behavior first and confirm reviewers understand the decision.

• Some governance capabilities and settings evolve; the portal blades and Graph beta endpoints change.

Steps are flagged where a name or path may drift — verify against current Microsoft guidance.

2. Requirements Guide — Entra ID Governance: Access Reviews & Entitlement Management

Verify **every** requirement before you configure governance. Each entry: **Description · Why required · How to verify · How to implement if missing.** The category-wide prerequisites (an active Entra ID tenant, base licensing, administrative roles, and break-glass accounts) are defined once in `global-requirements-entra-id-solutions.md` — confirm those first. The items below are **specific to Identity Governance**.

2.1 Entra ID Governance / P2 licensing for the users in scope

Description: Entitlement management, access reviews, and lifecycle workflows require **Microsoft Entra ID Governance** (features formerly under **Entra ID P2**). Lifecycle workflows are specifically part of the **Entra ID Governance** SKU. **Why required:** Without the correct license the Identity Governance blades appear but creating packages, reviews, or workflows is blocked, and licensing applies to the **users who use** the features (requestors, reviewers, workflow-targeted users). **How to verify:** Microsoft 365 admin center → **Billing** → **Licenses**; confirm **Entra ID Governance** (or P2) coverage. In the Entra admin center, **Identity governance** blades load without a licensing banner. Run `ENTIGA-Discovery.ps1` to confirm

the governance endpoints respond. **How to implement if missing:** Assign Entra ID Governance / P2 (often via EMS E5 or an add-on) to the in-scope users. *Verify current licensing rules against Microsoft's documentation — SKU coverage changes.*

2.2 Identity Governance administrative role

Description: The operator holds a role that can administer entitlement management, access reviews, and lifecycle workflows — **Identity Governance Administrator** (least privilege) or Global Administrator for tenant-wide setup. **Why required:** Creating catalogs, packages, reviews, and workflows requires the governance admin role; day-to-day catalog work can be delegated to **catalog owners** without tenant-wide rights. **How to verify:** Entra → **Roles and administrators**; confirm **Identity Governance Administrator** on the operator. **How to implement if missing:** Assign **Identity Governance Administrator**; reserve Global Administrator for the initial setup only. Delegate catalog administration to resource owners.

2.3 The resources to govern exist (groups, apps, sites)

Description: The **groups**, **enterprise applications**, and **SharePoint sites** that access packages will grant must already exist and be identified. **Why required:** An access package references **resource roles** that must be present in its catalog; you cannot bundle access to a group or app that doesn't exist. **How to verify:** Entra → **Groups** / **Enterprise applications**; confirm the target objects. Run `ENTIGA-Discovery.ps1` to inventory current groups and app assignments. **How to implement if missing:** Create the groups/apps first, or scope the pilot to resources that already exist. Record each resource's exact name for the catalog.

2.4 A pilot group and named requestors / approvers / reviewers

Description: An **Entra security group** `ENTIGA-Pilot` with a handful of test users, and named people to act as **approvers** (entitlement) and **reviewers** (access reviews). **Why required:** The pilot-first model targets `ENTIGA-Pilot`; approval and review flows need real identities to route to. **How to verify:** Entra → **Groups**; confirm `ENTIGA-Pilot` and its members. Confirm approver/reviewer identities exist. **How to implement if missing:** Create `ENTIGA-Pilot` (Assigned membership) and add test users; nominate approvers/reviewers (often the resource owner or the user's manager).

2.5 Manager attribute populated (for manager-as-approver / reviewer and mover workflows)

Description: The **Manager** attribute is set on in-scope users where you will use *manager* as the approver, *manager* as the self-review escalation, or a **mover** workflow keyed on manager change. **Why required:** "Manager" as a dynamic approver/reviewer and manager-based lifecycle logic resolve against the directory **Manager** relationship; if it is empty, routing fails or falls back. **How to verify:** Entra → **Users** → **(user)** → **Edit properties** → **Manager** is populated. Run `ENTIGA-Report.ps1` to list in-scope users missing a manager. **How to implement if missing:** Populate the Manager attribute (via HR sync, Entra Connect, or Graph) before relying on manager-based approval/review.

2.6 Employee lifecycle attributes for workflows (joiner/mover/leaver)

Description: For **lifecycle workflows**, the trigger attributes must be populated — `employeeHireDate` for joiner scheduling and `employeeLeaveDateTime` for leaver scheduling (typically written from an HR source such as Workday, SuccessFactors, or an HR-driven provisioning inbound flow). **Why required:** Scheduled joiner/leaver workflows fire relative to these dates; without them, only **on-demand** workflow runs are possible. **How to verify:** Entra → **Users** → **(user)** attributes, or Graph `employeeHireDate` / `employeeLeaveDateTime`. Run `ENTIGA-Lifecycle.ps1` to report which in-scope users have the trigger attributes set. **How to implement if missing:** Populate the attributes from the HR source (HR inbound provisioning or directory write), or plan to run workflows **on-demand** until the attribute feed exists.

2.7 Guest / external collaboration settings decided

Description: The **External collaboration settings** (who can invite guests, guest permissions) and, if used, **connected organizations** for external requestors are configured to support governed guest access. **Why required:** Guest governance (time-bound external access, guest access reviews, external requestor policies) depends on how guests enter the tenant and which external organizations are recognized. **How to verify:** Entra → **External Identities** → **External collaboration settings** and **Identity governance** → **Entitlement management** → **Connected organizations**. Run `ENTIGA-GuestReview.ps1` to inventory current guest accounts and their sign-in recency. **How to implement if missing:** Set external collaboration settings to your standard and add any **connected organizations** whose users should request packages, before publishing guest-facing packages.

2.8 Microsoft Graph PowerShell toolchain

Description: The **Microsoft Graph PowerShell SDK** installed on the operator's workstation, with consent for the least-privilege read scopes the scripts use. **Why required:** The five included tools authenticate via `Connect-MgGraph`; discovery, validation, and reporting run through Graph. **How to verify:** `Get-Module -ListAvailable Microsoft.Graph.Authentication` returns a version; running `ENTIGA-Discovery.ps1` connects successfully. **How to implement if missing:** `Install-Module Microsoft.Graph -Scope CurrentUser`; consent to `EntitlementManagement.Read.All`, `AccessReview.Read.All`, `LifecycleWorkflows.Read.All`, `User.Read.All` on first run.

Checklist

- Category global requirements confirmed (`global-requirements-entra-id-solutions.md`)
- Entra ID Governance / P2 licensing for in-scope users — *verify current SKU coverage*
- Identity Governance Administrator role held by the operator
- Target groups / apps / sites to govern exist and are named
- `ENTIGA-Pilot` group created; approvers and reviewers nominated
- Manager attribute populated where manager-based routing is used
- `employeeHireDate` / `employeeLeaveDateTime` populated (or on-demand plan) — `ENTIGA-Lifecycle.ps1`
- Guest / external collaboration + connected organizations decided — `ENTIGA-GuestReview.ps1`
- Microsoft Graph PowerShell SDK installed — `ENTIGA-Discovery.ps1`

Global Requirements — Entra ID Solutions (category-level)

These requirements apply to every solution in the Entra ID Solutions category. They are published once at the category level so individual listings stay small. Confirm these before deploying any identity solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *The identity plane is high risk. A mistake here breaks sign-in for everyone, so every change is staged and a break-glass account is kept ready before you touch anything.*

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID tenant. Why: It is the cloud identity backbone every other identity service builds on. Verify: Sign in to `https://entra.microsoft.com` as an administrator. Implement: Create a Microsoft 365 or Azure subscription, which provisions a tenant.

G2. Entra ID licensing (P1, P2 where noted)

Description: Microsoft Entra ID P1 (or P2 for risk-based and PIM features). Why: Hybrid features such as seamless sign-in, password writeback, and Conditional Access require P1+. Verify: M365 admin center -> `Billing -> Licenses`; confirm Entra ID P1/P2 (often bundled in M365 E3/E5 or EMS). Implement: Assign M365 E3/E5, EMS E3/E5, or standalone Entra ID P1/P2.

G3. Administrative roles

Description: Hybrid Identity Administrator and Global Administrator for the operator. Why: Configuring directory synchronization and authentication methods needs these elevated roles. Verify: Entra -> `Roles and administrators`; confirm `Hybrid Identity Administrator` (and a Global Administrator for the one-time first-sync consent). Implement: Assign `Hybrid Identity Administrator`; reserve `Global Administrator` for tenant-wide and first-sync changes.

G4. Break-glass (emergency access) accounts

Description: At least two cloud-only emergency access accounts, excluded from Conditional Access and MFA-broad policies. Why: If sync, federation, or an auth-method change locks out admins, break-glass accounts get you back in. Verify: Entra -> `Users`; confirm two cloud-only accounts (`@onmicrosoft.com`) with strong stored credentials and documented use. Implement: Create two cloud-only Global Administrator accounts; exclude them from policies; store credentials securely and monitor sign-ins.

G5. Healthy on-premises Active Directory

Description: A functional AD forest/domain with healthy replication and DNS. Why: Synchronization reads from AD; replication or DNS faults corrupt or stall the sync. Verify: Run `dcdiag` and `repadmin /replsummary` on a domain controller with no errors; run `HYBID-Discovery.ps1`. Implement: Resolve AD replication, DNS, and time-sync issues before synchronizing.

G6. An Entra Connect host server

Description: A domain-joined Windows Server (2016+), patched, for Microsoft Entra Connect Sync (or a lightweight host with the Cloud Sync agent). Why: Connect Sync runs as a service on this host and must reach both AD and Entra ID. Verify: A supported, domain-joined server with TLS 1.2, .NET, and PowerShell available; run `HYBID-Discovery.ps1`. Implement: Provision a dedicated server (not a domain controller for production), patch it, and enable TLS 1.2.

G7. UPN / domain hygiene

Description: User principal names use a public, verified, routable domain (not `.local`). Why: A non-routable UPN suffix produces sign-in problems and `onmicrosoft.com` fallbacks in the cloud. Verify: Entra -> `Identity -> Settings -> Domain names` shows the public domain Verified; AD UPN suffixes match it; run `HYBID-Discovery.ps1`. Implement: Add and verify the public domain in Entra; add the matching UPN suffix in AD and update user UPNs.

G8. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS (443) from the Connect host to Entra ID, plus required Microsoft 365 service URLs. Why: Synchronization, authentication-agent traffic, and tokens depend on these endpoints. Verify: From the Connect host, reach `https://login.microsoftonline.com` and the Microsoft 365 published URL/IP ranges. Implement: Allow the Microsoft 365 / Entra service URLs and IP ranges through the proxy/firewall.

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Entra ID Governance: Access Reviews & Entitlement Management

Follow these steps **in order**, in the **Microsoft Entra admin center** (`https://entra.microsoft.com` → **Identity governance**). Names in `code` are the exact objects to create — keep them consistent so the scripts, testing, and troubleshooting all refer to the same objects. Portal blade names evolve; if a label differs, match by intent and verify against current Microsoft guidance.

Object naming convention: `ENTIGA – <Purpose> – <Scope> – <Stage>`. The objects you will build:

Object	Name
Governance catalog	`ENTIGA – Catalog – Corp`
Access package	`ENTIGA – Package – LOBApp – Prod`
Assignment policy (employees)	`ENTIGA – Policy – Employees – Approval`
Assignment policy (guests)	`ENTIGA – Policy – Guests – TimeBound`
Access review (group)	`ENTIGA – Review – PrivGroup – Quarterly`
Access review (app)	`ENTIGA – Review – LOBApp – Quarterly`
Access review (guests)	`ENTIGA – Review – AllGuests – Quarterly`

Object	Name
Joiner workflow	`ENTIGA – Joiner – Onboarding`
Leaver workflow	`ENTIGA – Leaver – Offboarding`
Pilot group	`ENTIGA-Pilot`

> ***Model:*** *Assess → Configure → Pilot → Validate → Expand. Build each object, target* `ENTIGA-Pilot` ***first****, prove the gates in §5, then expand. Every production change has a rollback (see end of file).*

Step 1 — Assess and baseline the current state

1. Run `ENTIGA-Discovery.ps1` — inventories existing catalogs, access packages, policies, access review definitions, and lifecycle workflows so you know your starting posture (likely empty on a new tenant).

1. Run `ENTIGA-GuestReview.ps1` — lists current guest accounts with last sign-in, to size the guest governance problem.

1. Confirm the pilot group `ENTIGA-Pilot` exists (Entra → **Groups**) with test users and, ideally, one test **guest** account.

Step 2 — Create the governance catalog

1. **Identity governance** → **Entitlement management** → **Catalogs** → **+ New catalog**.

2. **Name:** `ENTIGA – Catalog – Corp`. **Description:** a short purpose line. **Enabled:** Yes.

Enabled for external users: Yes (required so guests can request packages from this catalog). **Create**.

1. Open the catalog → **Resources** → **+ Add resources** → add the resource(s) the pilot package will grant:

• a **Group** (e.g. an app-access security group),

• an **Application** (an enterprise app), and/or

• a **SharePoint site**.

Only resources added to the catalog can be bundled into its packages.

Step 3 — Create the pilot access package

1. In `ENTIGA – Catalog – Corp` → **Access packages** → **+ New access package**.

2. **Basics:** Name `ENTIGA – Package – LOBApp – Prod`; description of what access it grants.

3. **Resource roles:** add the role(s) users receive — e.g. **Member** of the group, a specific **app role**, **and/or a** site** role from the catalog's resources.

1. Leave policy creation for the next step. **Create**.

Step 4 — Add the employee assignment policy (request + approval + expiry)

1. Open `ENTIGA – Package – LOBApp – Prod` → **Policies** → **+ Add policy**.
2. **Name:** `ENTIGA – Policy – Employees – Approval`.
3. **Users who can request: For users in your directory** → **Specific users and groups** → add `ENTIGA-Pilot` (pilot-first; you will add the broader audience at Expand).
1. **Approval: Require approval = Yes**; **Stage 1 approver = Manager** (or a named approver / the resource owner); require **justification**. (Add a second stage only if your process needs it.)
1. **Requestor information:** optionally add a question (e.g. business justification).
2. **Lifecycle:** set **Expiration** → **Access expires** → **After number of days** (e.g. **90**) so the grant is **time-bound**; enable **Require access reviews** on renewal if you want the package itself reviewed.
1. **Create.** This is the core "request → approve → time-bound grant → expire" loop.

Step 5 — Add the guest (time-bound external) assignment policy

1. Same package → **Policies** → **+ Add policy**.
2. **Name:** `ENTIGA – Policy – Guests – TimeBound`.
3. **Users who can request: For users not in your directory** → choose **Specific connected organizations **(from Requirement 2.7) or** All configured connected organizations**.
1. **Approval: Require approval = Yes**; approver = an **internal sponsor** (a named internal approver — external requestors must have an internal approver).
1. **Lifecycle:** set a **shorter expiry** for guests (e.g. **30 days**) so external access is strictly time-bound, and enable **Require access reviews** so continued guest access is re-certified.
1. **Create.** Guests now request via **My Access** and receive **time-bound** access only.

Step 6 — Create the recurring access reviews

Create three reviews under **Identity governance** → **Access reviews** → **+ New access review**.

6a — Group membership review

1. **Review type: Teams + Groups** → **Select Teams + groups** → pick a privileged/sensitive group.
2. **Scope:** All users (or Guest users only for a guest-focused group).
3. **Name:** `ENTIGA – Review – PrivGroup – Quarterly`. **Recurrence:** Quarterly.
4. **Reviewers: Group owner(s)** or **Manager of the user** (or named reviewers).
5. **Settings: Auto apply results = Enabled**; **If reviewers don't respond** → **Remove access** (or **No change** while piloting — see the note); enable **Justification required** and reviewer **recommendations**. **Create.**

6b — Application assignment review

1. **Review type: Applications** → **Select applications** → the enterprise app.

2. **Name:** `ENTIGA – Review – LOBApp – Quarterly`. Reviewers = app owner / manager. Same auto-apply

settings. **Create.**

6c — Guest access review

1. **Review type:** review **guest users' access** — scope to **Guest users** across the target

group(s)/app(s) or across the tenant per your standard.

1. **Name:** `ENTIGA – Review – AllGuests – Quarterly`. **Reviewers:** manager or the guest's **sponsor**;

optionally enable **self-review** with manager fallback. **Auto apply = Enabled**; don't-respond → **Remove access**. **Create.**

> ***Pilot safely:*** *for the* ***first*** *run set "if reviewers don't respond" to* ***No change*** *and review the decisions manually, so an unanswered review doesn't remove access before reviewers are trained. Switch to* ***Remove access*** *once the process is trusted.*

Step 7 — Create the lifecycle workflows (joiner / leaver)

Under **Identity governance** → **Lifecycle workflows** → **Workflows** → **+ Create a workflow**.

7a — Joiner (onboarding)

1. **Choose a template: Onboard pre-hire employee** (or **New hire**). **Next.**
2. **Basics:** Name `ENTIGA – Joiner – Onboarding`.
3. **Execution conditions: Trigger = days relative to** `employeeHireDate` (e.g. **0 days** on hire);

Scope: a rule that resolves to `ENTIGA-Pilot` members (e.g. filter on a pilot attribute) for the pilot.

1. **Tasks:** keep template tasks such as **Enable user account**, **Send welcome email**, **Add user to

groups**, and (optionally)** Request access package assignment** for `ENTIGA – Package – LOBApp – Prod`.

1. **Review + create.** Leave it enabled but scoped to the pilot only.

7b — Leaver (offboarding)

1. **+ Create a workflow** → **Choose template: Real-time employee termination** (on-demand) or

Post-offboarding / Scheduled leaver keyed to `employeeLeaveDateTime`.

1. **Name:** `ENTIGA – Leaver – Offboarding`. **Scope:** `ENTIGA-Pilot`.
2. **Tasks: Remove user from all groups**, **Remove user from all Teams**, **Remove all access

package assignments**, Disable user account**, Revoke all refresh tokens (sessions)**.

1. **Review + create.** For the pilot, run this **on-demand** against a test leaver rather than waiting

for a real termination.

> ***Mover pattern:*** *a* ***mover*** *workflow uses the same builder with a trigger on* ***attribute change*** *(e.g. department/manager change) and tasks that* ***remove stale group memberships*** *and* ***request the new role's package****. Build it with the same convention (*`ENTIGA – Mover – DeptChange`*) once*

joiner and leaver are validated.

Step 8 — Validate the pilot (the gates)

Run `ENTIGA-Validate.ps1` and confirm on `ENTIGA-Pilot`:

• **G1 Request/approve/expire** — a pilot user requests `ENTIGA – Package – LOBApp – Prod` in **My

Access**, the approver approves, access is granted** and shows an expiry**.

• **G2 Review/remove** — a review lists members; a **Deny** decision with **auto-apply** actually **removes** the access.

• **G3 Guest time-bound** — a guest requests via the guest policy, is approved by the internal sponsor, and receives access with the **shorter expiry**.

• **G4 Lifecycle** — the joiner workflow runs its tasks; the leaver workflow (on-demand) **disables** the test account and **revokes** sessions.

Pass = all four gates green. **Fail** = fix before expanding (see §6).

Step 9 — Expand to production in waves

1. In each assignment policy, **broaden the requestor scope** from `ENTIGA-Pilot` to the intended audience (specific groups or **All members**), one package/audience at a time.

1. Add more **resources** to the catalog and publish additional **access packages** using the same convention.

1. **Broaden review scope** from the pilot group/app to the full set; switch don't-respond behavior to **Remove access** once trusted.

1. **Broaden workflow scope** from `ENTIGA-Pilot` to the full population and enable **scheduled** (attribute-driven) execution.

1. Re-run `ENTIGA-Validate.ps1` and `ENTIGA-Report.ps1` after each wave to capture evidence.

> ***Rollback (any step):*** *disable or narrow the offending object rather than deleting it — set an* ***assignment policy*** *back to* `ENTIGA-Pilot` *only; set a review's don't-respond to* ***No change*** *or stop the review;* ***disable*** *a lifecycle workflow (****Workflows*** *→* ***(workflow)*** *→* ***Disable****). To restore an access removed by auto-apply, re-request/re-approve the package or re-add the membership.*
> *Lifecycle-workflow tasks that* ***disable accounts / revoke sessions*** *are high-impact — treat leaver changes under change control.*

Order of operations

```
Assess (Discovery/GuestReview) → catalog + resources → access package →
employee policy → guest policy → 3 access reviews → joiner + leaver workflows
→ validate the 4 gates → expand in waves
```

5. Testing & Validation — Entra ID Governance: Access Reviews & Entitlement Management

Validate on the **pilot** (`ENTIGA-Pilot`) first. Each test has a **pass/fail** signal. Do not expand until all four gates pass. Reference scripts are named in each row.

Test matrix

#	Gate	What you do	Pass signal	Fail signal
1	**G1 Request**	A pilot user opens **My Access** (`https://myaccess.microsoft.com`) and requests `ENTIGA – Package – LOBApp – Prod`	Request is submitted and routed to the approver	Package not visible / not requestable
2	**G1 Approve**	The approver (Manager / named) approves in **My Access** → **Approvals**	Access is granted; the resource role is assigned	Approval never arrives / grant not applied
3	**G1 Expiry**	Inspect the assignment after approval	Assignment shows a **time-bound expiry** (e.g. 90 days)	Grant is permanent / no expiry
4	**G2 Review runs**	Start `ENTIGA – Review – PrivGroup – Quarterly`; reviewer opens **My Access** → **Access reviews**	Reviewer sees members + recommendations and can Approve/Deny	Review not started / reviewer sees nothing
5	**G2 Auto-apply removes**	Reviewer **Denies** a test member; auto-apply runs	The denied member is **removed** from the group	Access remains after a Deny decision
6	**G3 Guest time-bound**	A test guest requests via `ENTIGA – Policy – Guests – TimeBound`; internal sponsor approves	Guest gets access with the **shorter expiry** (e.g. 30 days)	Guest gets permanent access / no approver
7	**G4 Joiner**	Run `ENTIGA – Joiner – Onboarding` on-demand for a test user	Tasks run (account enabled, added to groups, welcome sent)	Workflow fails / tasks skipped

#	Gate	What you do	Pass signal	Fail signal
8	**G4 Leaver**	Run `ENTIGA – Leaver – Offboarding` on-demand for a test leaver	Account **disabled**, group/package assignments **removed**, sessions **revoked**	Access/sessions remain after run

Automated checks

• `ENTIGA-Discovery.ps1` — inventories catalogs, access packages + policies, access review definitions, and lifecycle workflows so you can confirm the named objects exist with the expected configuration.

• `ENTIGA-Validate.ps1` — checks each **named** object from §3 is present and correctly wired (package has an approval policy with an expiry; each review has auto-apply set; each workflow is enabled and scoped) and **flags gaps** — the automated form of the gate table.

• `ENTIGA-Report.ps1` — exports **audit/recertification evidence**: current access package assignments (who has what, granted/expiry dates) and recent access review decisions.

• `ENTIGA-GuestReview.ps1` — lists guests with **last sign-in** so you can confirm stale guests are being caught by the guest review.

• `ENTIGA-Lifecycle.ps1` — reports workflow **run history** and which in-scope users have the trigger attributes (`employeeHireDate` / `employeeLeaveDateTime`) populated.

Evidence to capture (for audit / recertification)

• Screenshots: the package in **My Access**, an approval decision, an assignment showing **expiry**.

• A completed **access review** with decisions and the **auto-apply** result.

• `ENTIGA-Report.ps1` CSV of assignments + review decisions (the recertification artifact).

• Lifecycle workflow **run report** showing joiner/leaver task outcomes.

Exit criteria (before Expand)

• G1 — request → approve → **time-bound** grant works end to end

• G2 — a Deny decision with **auto-apply** actually removes access

• G3 — guest access is granted with a **shorter expiry** via an internal sponsor

• G4 — joiner runs its tasks; leaver **disables + revokes + removes** assignments

• `ENTIGA-Validate.ps1` shows no gaps for the pilot objects

• `ENTIGA-Report.ps1` produces a clean assignment + decision evidence export

6. Troubleshooting — Entra ID Governance: Access Reviews & Entitlement Management

Symptom → likely root cause → fix. Start with `ENTIGA-Discovery.ps1` (are the named objects present and wired?) and `ENTIGA-Validate.ps1` (which gate is failing?) — most governance issues are a missing policy scope, a licensing gap, or an empty attribute the flow depends on.

The Identity Governance blades show a licensing banner / creation is blocked

- **Root cause:** the operator or the in-scope users lack **Entra ID Governance / P2** licensing;

lifecycle workflows need the **Entra ID Governance** SKU specifically.

- **Fix:** assign the correct license (M365 admin center → **Billing** → **Licenses**) to the operator and to

requestors/reviewers/workflow-targeted users. Re-open the blade. *Verify current SKU coverage against Microsoft's documentation — it changes.*

An access package isn't requestable in My Access

- **Root cause:** no assignment **policy** targets the requestor, the requestor isn't in the policy's

scope (`ENTIGA-Pilot` only during pilot), or the catalog isn't **enabled for external users** (for a guest requestor).

- **Fix:** confirm `ENTIGA – Policy – Employees – Approval` scopes the user (or the group they're in);

for guests, confirm the catalog is **enabled for external users** and `ENTIGA – Policy – Guests – TimeBound` allows their connected organization. `ENTIGA-Validate.ps1` reports policy scope.

Approval never arrives / request sits pending

- **Root cause:** the approver is **Manager** but the requestor has **no manager** set; or the named

approver is disabled; or the approver isn't checking **My Access** → **Approvals**.

- **Fix:** populate the **Manager** attribute (Requirement 2.5 — `ENTIGA-Report.ps1` lists users missing

one), or set a **named/fallback approver**. Confirm the approver acts in **My Access**, not email alone.

A granted assignment has no expiry (access is permanent)

- **Root cause:** the assignment policy's **Lifecycle** → **Expiration** was left at **Never**.
- **Fix:** edit the policy → **Lifecycle** → set **Access expires** → **After N days**; enable

Require access reviews on renewal. Existing permanent grants can be ended and re-requested under the time-bound policy.

An access review Deny didn't remove access

• **Root cause: Auto apply results** is **Disabled**, so decisions are recorded but not enforced; or the review is still in progress; or "if reviewers don't respond" was set to **No change** and nobody acted.

• **Fix:** edit the review → **Settings** → **Enable auto apply results**. Confirm the review has **ended** (decisions apply at cycle end). `ENTIGA-Validate.ps1` flags reviews without auto-apply.

An access review removed access it shouldn't have

• **Root cause:** "if reviewers don't respond" was set to **Remove access** on the **first** cycle and reviewers didn't respond in time.

• **Fix:** for early cycles set don't-respond to **No change** and review manually; switch to **Remove access** once reviewers are trained. Restore the removed access by re-requesting/re-approving the package or re-adding the membership.

A guest can't request the guest package

• **Root cause:** the catalog isn't **enabled for external users**, the guest's organization isn't a recognized **connected organization**, or `ENTIGA – Policy – Guests – TimeBound` scopes a different set of external users.

• **Fix:** enable the catalog for external users; add the guest's domain as a **connected organization** (Requirement 2.7); confirm the guest policy's requestor scope. `ENTIGA-GuestReview.ps1` shows current guests and their source.

A lifecycle workflow didn't fire on schedule

• **Root cause:** the trigger attribute (`employeeHireDate` / `employeeLeaveDateTime`) is empty or in the wrong format, the user isn't in the workflow **scope**, or the workflow is **disabled**.

• **Fix:** `ENTIGA-Lifecycle.ps1` reports which in-scope users have the trigger attributes populated. Populate the attribute from the HR source, confirm the scope rule resolves the user, and ensure the workflow is **enabled**. Test **on-demand** first.

A leaver ran but access/sessions remained

• **Root cause:** the leaver workflow is missing the **remove-assignments / disable / revoke-sessions** tasks, or the user's access came from a source the tasks don't cover (e.g. a directly-assigned app role outside a package).

• **Fix:** confirm `ENTIGA – Leaver – Offboarding` includes **Remove from all groups/Teams**, **Remove all access package assignments**, Disable account**, and** Revoke all refresh tokens**. Access granted outside entitlement management must be removed by its own mechanism (or brought under a package).

Editing / rebuilding a package broke existing grants

• **Root cause:** removing a **resource role** from a package or deleting a policy can affect existing

assignments that depend on it.

• **Fix:** change packages additively where possible; when a resource role must change, communicate and

re-issue assignments. Keep the pilot separate from production packages so experiments don't touch live access.

First moves for any governance issue

1. `ENTIGA-Discovery.ps1` — are the named catalog/package/policy/review/workflow objects present?

2. `ENTIGA-Validate.ps1` — which gate is failing (missing expiry, no auto-apply, disabled workflow)?

3. `ENTIGA-Lifecycle.ps1` / `ENTIGA-Report.ps1` — are the driving attributes (manager, hire/leave dates)

populated for the affected users?

1. Only then change configuration — preview any remediation with `-WhatIf`, and keep the current state as

your rollback.

7. Best Practices — Entra ID Governance: Access Reviews & Entitlement Management

Entitlement management (catalogs & packages)

• **Delegate with catalogs.** Give resource owners **catalog owner** rights on their own catalog instead

of tenant-wide governance roles — least privilege, and the people who know the access own the packages.

• **Package the job, not the resource.** Build access packages around a **role/task** ("Sales LOB

access") that bundles the groups/apps/sites needed, so one request grants everything — cleaner than one package per group.

• **Always time-bound.** Set an **expiry** on every assignment policy; permanent access is how sprawl

returns. Use **review-on-renewal** so continued access is re-justified, not auto-extended.

• **Separate employee and guest policies** on the same package — different requestor scope, approver, and

a **shorter expiry for guests**. Never let external users self-approve; require an **internal sponsor**.

Access reviews

• **Auto-apply, but pilot the fallback.** Enable **auto-apply** so decisions actually take effect, but on

the first cycle set "if reviewers don't respond" to **No change** and review manually; switch to **Remove access** once reviewers are trained. An unanswered review shouldn't silently strip access on day one.

- **Right reviewer, right cadence.** Group/app owners review their own resources; managers review their

reports; guests are reviewed by their **sponsor**. Match frequency to sensitivity — privileged and guest access quarterly (or more), routine access semi-annually.

- **Show recommendations and require justification** so reviewers make informed, defensible decisions —

and so the decision record is audit-quality.

Lifecycle workflows

- **Start on-demand, then schedule.** Prove joiner/leaver tasks by running them **on-demand** against

test users before enabling **attribute-driven** (`employeeHireDate` / `employeeLeaveDateTime`) scheduling for the whole population.

- **Leaver = disable + revoke + remove.** An offboarding workflow should **remove group/package

assignments**, disable the account**, and** revoke sessions** — deprovisioning that leaves standing access or live tokens is the classic gap.

- **Keep tasks idempotent and scoped.** Scope every workflow tightly (start with `ENTIGA-Pilot`) and keep

high-impact tasks (disable/revoke) in the **leaver** workflow only.

Guest governance

- **Front-door guests through packages.** Prefer governed **access package requests** (with an internal

sponsor and expiry) over ad-hoc guest invites, so every external grant is time-bound and reviewable.

- **Review stale guests routinely.** Run `ENTIGA-GuestReview.ps1` to surface guests with old/absent

sign-ins and feed them into the guest access review — remove those no longer needed.

Change safety

- **Pilot-first, always** — prove the four gates on `ENTIGA-Pilot` before expanding.
- **Auto-apply and leaver tasks change real access** — treat them like production changes: staged

rollout, a documented rollback, and (for leaver) a change window.

- **Keep a rollback ready** — narrow a policy back to the pilot, set a review to **No change**, or

disable a workflow to stop it; re-request/re-approve to restore removed access.

- **Least privilege for the operators** — run day-to-day governance as **Identity Governance

Administrator**; reserve Global Administrator for initial setup only.

Hygiene

• **No secrets in automation.** Scripts authenticate interactively via `Connect-MgGraph` with least

read scopes — never embed credentials or tokens.

• **Document per package:** resources, approver, expiry, and the review that recertifies it.

Where to go next (adjacent catalog areas)

• **Entra ID Governance — PIM & Access Reviews** — just-in-time **privileged role** activation and

role-assignment reviews (this solution governs resource/app access; PIM governs directory roles).

• **Zero Trust — Conditional Access Baseline** — enforce the sign-in conditions under which any of this

access can be used.

• **Enterprise App & Service Principal Governance** — govern the apps and workload identities that access

packages grant into.

8. Deliverables Summary — Entra ID Governance: Access Reviews & Entitlement Management

What you receive

A complete, self-guided package to deploy **Microsoft Entra ID Governance** — entitlement management (catalogs, access packages, approval + lifecycle policies), recurring access reviews (groups, apps, guests), and joiner/mover/leaver lifecycle workflows — so access is requested, approved, time-bound, reviewed, and revoked automatically, with audit-ready evidence.

Documentation

• **Executive Summary** — the problem (access sprawl), the solution (three governance pillars), the

outcome, the boundary, and the licensing gate.

• **Solution Overview** — how entitlement management, access reviews, and lifecycle workflows fit

together, the building blocks (catalog → package → policy; review definition; workflow → tasks), scope, and what's out of scope (PIM is a separate solution).

• **Requirements Guide** — governance-specific prerequisites (Entra ID Governance/P2 licensing, the

Identity Governance role, resources to govern, pilot group + approvers/reviewers, manager and HR lifecycle attributes, guest/external settings, the Graph toolchain), each with Verify / Implement, on top of the category global requirements.

• **Implementation Guide** — ordered steps: assess → catalog + resources → access package → employee

policy → guest policy → three access reviews → joiner + leaver workflows → validate → expand, with **every object named** using the `ENTIGA – <Purpose> – <Scope> – <Stage>` convention.

• **Testing & Validation** — the four gates (request/approve/expire, review/remove, guest time-bound, lifecycle) as a pass/fail matrix, automated checks, and the evidence to capture for recertification.

• **Troubleshooting** — symptom → root-cause → fix for the common failures (package not requestable, approval stuck, auto-apply not removing, guest can't request, workflow not firing).

• **Best Practices** — catalog delegation, time-bound packages, safe auto-apply, leaver deprovisioning, guest governance, change safety, and where to go next.

PowerShell tools (Microsoft Graph) — safe by default

Script	Purpose	Default mode
`ENTIGA-Discovery.ps1`	Inventory catalogs, access packages + policies, access review definitions, and lifecycle workflows (baseline posture)	Read-only
`ENTIGA-Validate.ps1`	Check each named §3 object exists and is wired (package has approval + expiry; reviews auto-apply; workflows enabled/scoped); flag gaps	Read-only
`ENTIGA-Report.ps1`	Export audit/recertification evidence — access package assignments (who/when/expiry) + recent access review decisions	Read-only
`ENTIGA-GuestReview.ps1`	Report guest accounts with last sign-in and access; preview stale-guest cleanup	Read / `-WhatIf`
`ENTIGA-Lifecycle.ps1`	Report workflow run history + which users have `employeeHireDate` / `employeeLeaveDateTime`; preview an on-demand run	Read / `-WhatIf`

All scripts use `Connect-MgGraph` with **least scopes**, contain **no credentials or secrets**, and any change/remediation defaults to a `-WhatIf` **preview** gated behind an explicit `-Commit`.

Outcomes

- Access is **requested and approved**, not assigned by hand — and it **expires**.
- Group, app, and **guest** access is **recertified on a schedule**, with removals that actually apply.
- Joiner/mover/leaver tasks are **automated** — new hires equipped, leavers fully deprovisioned.
- Every grant has a **request, approver, and review decision** on record — audit-ready.
- A **pilot-first**, gated, reversible rollout with a documented rollback for every change.

Assumptions & boundaries

- **Entra ID Governance / P2 licensing** covers the in-scope users — *verify current SKU coverage*.
- The **resources** to govern (groups/apps/sites) already exist.
- HR **lifecycle attributes** feed scheduled workflows; without them, workflows run **on-demand**.
- **PIM** (privileged role activation) is a **separate adjacent solution**; this governs resource/app access.
- **Which** resources, cadences, approvers, and reviewers to use are the customer's governance decisions.

*Delivered as a **Complete Solution Package (PDF + Word + scripts)** — the full documentation set plus all five PowerShell tools in a single ZIP, yours to keep and reuse on every rollout.*

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

ENTIGA-Discovery.ps1

Purpose: Baselines the current governance posture: entitlement-management catalogs, access packages and their assignment policies, access review definitions, and lifecycle workflows. Run before you configure anything to see your starting point (usually near-empty on a new tenant). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTIGA-Discovery.ps1 -CsvFolder C:\Temp\entiga Requires the Microsoft Graph SDK and the read scopes below (consented on first run). EXPECTED RESULTS Console tables for catalogs, packages+policies, access reviews, and workflows, plus optional CSVs. PASS = the objects you built (ENTIGA - *) appear with the expected configuration. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Discovery.ps1 > [CmdletBinding()] param([string]$CsvFolder)

How to run: powershell -ExecutionPolicy Bypass -File .\ENTIGA-Discovery.ps1 -CsvFolder C:\Temp\entiga Requires the Microsoft Graph SDK and the read scopes below (consented on first run). EXPECTED RESULTS Console tables for catalogs, packages+policies, access reviews, and workflows, plus optional CSVs. PASS = the objects you built (ENTIGA - *) appear with the expected configuration. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Discovery.ps1 > [CmdletBinding()] param([string]$CsvFolder)

ENTIGA-GuestReview.ps1

Purpose: Sizes and governs guest (external) access. Lists every guest with creation date, last interactive sign-in, and pending-acceptance state, and flags guests idle beyond a threshold as stale-access candidates to feed into 'ENTIGA - Review - AllGuests - Quarterly'. By default it only REPORTS. With -Commit it would disable stale guests; -WhatIf (the default when -Commit is set without confirmation) previews exactly which accounts would be affected — nothing is changed without an explicit -Commit. HOW TO RUN Report only: .\ENTIGA-GuestReview.ps1 -StaleDays 90 -CsvPath C:\Temp\guests.csv Preview change:.\ENTIGA-GuestReview.ps1 -StaleDays 90 -Commit -WhatIf Apply (gated): .\ENTIGA-GuestReview.ps1 -StaleDays 90 -Commit # disables stale guests EXPECTED RESULTS A guest inventory table + a stale-guest list. PASS = no unexpected/idle guests linger. .NOTES White-label — Entra ID Solutions. Read-only by default; any change is gated behind -Commit with a -WhatIf preview. No credentials stored. Disabling (not deleting) is used so it is reversible. .EXAMPLE .\ENTIGA-GuestReview.ps1 -StaleDays 60 > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([int]$StaleDays = 90, [string]$CsvPath, [switch]$Commit)

How to run: Report only: .\ENTIGA-GuestReview.ps1 -StaleDays 90 -CsvPath C:\Temp\guests.csv Preview change:.\ENTIGA-GuestReview.ps1 -StaleDays 90 -Commit -WhatIf Apply (gated): .\ENTIGA-GuestReview.ps1 -StaleDays 90 -Commit # disables stale guests EXPECTED RESULTS A guest inventory table + a stale-guest list. PASS = no unexpected/idle guests linger. .NOTES White-label — Entra ID Solutions. Read-only by default; any change is gated behind -Commit with a -WhatIf preview. No credentials stored. Disabling (not deleting) is used so it is reversible. .EXAMPLE .\ENTIGA-GuestReview.ps1 -StaleDays 60 > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([int]$StaleDays = 90, [string]$CsvPath, [switch]$Commit)

ENTIGA-Lifecycle.ps1

Purpose: Supports the joiner/mover/leaver workflows. Reports (1) each ENTIGA workflow's recent run history and task outcomes, and (2) which in-scope users have the scheduling attributes employeeHireDate / employeeLeaveDateTime populated (without them only on-demand runs work). With -RunUserId it will trigger an ON-DEMAND run of a named workflow for a test user — but only as a -WhatIf preview unless -Commit is given, so you never fire a real onboarding/offboarding by accident. HOW TO RUN Report: .\ENTIGA-Lifecycle.ps1 Attr check: .\ENTIGA-Lifecycle.ps1 -CsvPath C:\Temp\lifecycle-attrs.csv Preview run: .\ENTIGA-Lifecycle.ps1 -WorkflowName 'ENTIGA - Joiner - Onboarding' -RunUserId <guid> -WhatIf Apply run: .\ENTIGA-Lifecycle.ps1 -WorkflowName 'ENTIGA - Joiner - Onboarding' -RunUserId <guid> -Commit EXPECTED RESULTS Workflow run table + attribute-readiness summary. PASS = workflows run clean and target users have the trigger attributes (or you run on-demand deliberately). .NOTES White-label — Entra ID Solutions. Read-only by default; on-demand runs gated behind -Commit with a -WhatIf preview. No credentials stored. .EXAMPLE .\ENTIGA-Lifecycle.ps1 > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$WorkflowName, [string]$RunUserId, [string]$CsvPath, [switch]$Commit)

How to run: Report: .\ENTIGA-Lifecycle.ps1 Attr check: .\ENTIGA-Lifecycle.ps1 -CsvPath C:\Temp\lifecycle-attrs.csv Preview run: .\ENTIGA-Lifecycle.ps1 -WorkflowName 'ENTIGA - Joiner - Onboarding' -RunUserId <guid> -WhatIf Apply run: .\ENTIGA-Lifecycle.ps1 -WorkflowName 'ENTIGA - Joiner - Onboarding' -RunUserId <guid> -Commit EXPECTED RESULTS Workflow run table + attribute-readiness summary. PASS = workflows run clean and target users have the trigger attributes (or you run on-demand deliberately). .NOTES White-label — Entra ID Solutions. Read-only by default; on-demand runs gated behind -Commit with a -WhatIf preview. No credentials stored. .EXAMPLE .\ENTIGA-Lifecycle.ps1 > [CmdletBinding(SupportsShouldProcess=$true, ConfirmImpact='High')] param([string]$WorkflowName, [string]$RunUserId, [string]$CsvPath, [switch]$Commit)

ENTIGA-Report.ps1

Purpose: Produces the recertification artifact auditors ask for: current access package assignments (who has what, when granted, when it expires) and the most recent access review decisions (who reviewed, the decision, and the justification). Also lists in-scope users missing a Manager attribute (needed for manager-as-approver / manager-based reviews). HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTIGA-Report.ps1 -CsvFolder C:\Temp\entiga-audit EXPECTED RESULTS Console summary + CSVs: assignments.csv, review-decisions.csv, missing-manager.csv. Keep the CSVs as recertification evidence. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Report.ps1 -CsvFolder C:\Temp\entiga-audit > [CmdletBinding()] param([string]$CsvFolder = ".\entiga-audit")

How to run: powershell -ExecutionPolicy Bypass -File .\ENTIGA-Report.ps1 -CsvFolder C:\Temp\entiga-audit EXPECTED RESULTS Console summary + CSVs: assignments.csv, review-decisions.csv, missing-manager.csv. Keep the CSVs as recertification evidence. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Report.ps1 -CsvFolder C:\Temp\entiga-audit > [CmdletBinding()] param([string]$CsvFolder = ".\entiga-audit")

ENTIGA-Validate.ps1

Purpose: The automated form of the pilot gate table. Confirms the objects from the implementation guide exist and are correctly configured: the access package has an approval policy WITH an expiry, each access review has auto-apply enabled, and each lifecycle workflow is enabled. Reports each check as

PASS / FAIL / WARN so you know exactly what to fix before expanding. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\ENTIGA-Validate.ps1 Optional: -Prefix 'ENTIGA' to match objects by naming convention (default 'ENTIGA'). EXPECTED RESULTS A checklist with PASS/FAIL per gate and a final count. PASS on all gates = ready to expand. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Validate.ps1 -Prefix 'ENTIGA' > [CmdletBinding()] param([string]$Prefix = 'ENTIGA')

How to run: powershell -ExecutionPolicy Bypass -File .\ENTIGA-Validate.ps1 Optional: -Prefix 'ENTIGA' to match objects by naming convention (default 'ENTIGA'). EXPECTED RESULTS A checklist with PASS/FAIL per gate and a final count. PASS on all gates = ready to expand. .NOTES White-label — Entra ID Solutions. Read-only against Microsoft Graph. No credentials stored. .EXAMPLE .\ENTIGA-Validate.ps1 -Prefix 'ENTIGA' > [CmdletBinding()] param([string]$Prefix = 'ENTIGA')